

NovaMart

Setup & Operations Guide

Laravel 11 · MySQL · Razorpay · Tailwind CSS

Created by **Piyush Pathar**

Version: 09 October 2026

For developers who install, configure and deploy the store, and for admins who run it.

Contents

1. Overview	3
Technology stack	3
Main features	3
2. Requirements	3
3. Local Setup	4
3.1 Install dependencies	4
3.2 Create the environment file	4
3.3 Create the database	4
3.4 Create tables and load sample data	4
3.5 Build front-end assets and start the server	4
3.6 Sample accounts (local only)	5
3.7 Sample coupons	5
4. How Configuration Works	5
5. Configuration Reference	5
5.1 Environment variables (.env)	5
5.2 Admin > Settings tabs	6
5.3 JSON settings format	6
6. Payments (Razorpay)	7
6.1 Mock mode (local testing)	7
6.2 Going live	7
6.3 Payment safety checks	7
7. Google Sign-In (optional)	7
8. Admin Panel Guide	8
9. Orders and Stock	8
10. Project Structure	9
11. Routes Reference	9
Storefront	9
Admin (login + admin role required)	10
12. Testing	10
13. Production Deployment	10
13.1 First deployment	10
13.2 Checklist	10
13.3 Updating an existing installation	11
14. Troubleshooting	11
15. Known Limitations	11

1. Overview

NovaMart is a multi-category online store (electronics, fashion, home, audio, footwear, beauty) built on Laravel 11 with a MySQL database. Customers browse, search, add to cart, apply coupons and pay with Razorpay or Cash on Delivery. Admins manage the catalogue, orders, coupons, users, content pages and the whole look of the storefront from a built-in admin panel, without code changes.

Technology stack

Layer	Technology
Backend	PHP 8.2+, Laravel 11
Database	MySQL 8+ (JSON columns used for product images, specs, order items, addresses)
Frontend	Blade templates, Tailwind CSS 3, Alpine.js, Lucide icons, built with Vite 6
Payments	Razorpay (UPI, cards, net banking) with a built-in mock mode for local testing; Cash on Delivery
Sign-in	Email + password; optional Google Sign-In (OAuth 2.0)
Sessions, cache, queue	Database drivers (no Redis required)

Main features

- Catalogue with categories, brands, faceted search, live search suggestions, ratings and verified-buyer reviews.
- Session cart that always uses current prices and stock; coupons (percentage or fixed, minimum spend, usage limits, expiry).
- Checkout with saved addresses (guests supported), standard or express shipping, Razorpay or Cash on Delivery.
- Stock is reserved when an order is placed and returned when an order is cancelled or refunded.
- Admin panel: dashboard, products, categories, orders and status tracking, coupons, users, CMS pages, store settings.
- Config-driven storefront: branding, menus, banners, homepage sections, footer, shipping fees and gateways are all editable in Admin > Settings.
- SEO: per-page meta tags, sitemap.xml and robots.txt.

2. Requirements

Software	Version	Notes
PHP	8.2 or newer	Extensions: pdo_mysql, mbstring, openssl, tokenizer, xml, ctype, json, bcmath, curl, fileinfo
Composer	2.x	PHP dependency manager
MySQL	8.0 or newer	MariaDB 10.6+ also works
Node.js / npm	Node 18+ (20 LTS recommended)	Only needed to build CSS/JS assets
Web server (production)	Nginx or Apache	Document root must point to the <code>public/</code> folder

3. Local Setup

Run these steps from the project folder. They were verified on a fresh, empty MySQL database.

3.1 Install dependencies

```
composer install
npm install
```

3.2 Create the environment file

```
cp .env.example .env
php artisan key:generate
```

Open `.env` and set your database credentials (see section 5.1 for every setting):

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=novamart_ecommerce
DB_USERNAME=root
DB_PASSWORD=your_password
```

3.3 Create the database

```
mysql -u root -p -e "CREATE DATABASE novamart_ecommerce
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci"
```

3.4 Create tables and load sample data

```
php artisan migrate --seed
```

This creates all tables and loads 45 sample products in 7 categories, 7 content pages (About, Contact, Shipping, Refund, Terms, Privacy, FAQ), 2 coupons, 2 user accounts, a sample order and default store settings.

Important: The seeder empties the users, products, orders and settings tables before loading sample data. Only run it on a new local database, never on a database with real data.

3.5 Build front-end assets and start the server

```
npm run build          # one-off build
php artisan serve      # http://127.0.0.1:8000
```

For development with live reloading, run `composer run dev` instead. It starts the web server, queue worker, log viewer and Vite together.

3.6 Sample accounts (local only)

Role	Email	Password
Admin	admin@novamart.com	Password@123
Customer	demo@novamart.com	Password@123

The admin panel is at `/admin`. The login page lists these test accounts only when `APP_ENV=local`.

3.7 Sample coupons

Code	Discount	Minimum order
NOVAMART10	10% off	INR 999
WELCOME50	INR 1,000 off	INR 5,000

4. How Configuration Works

Every store setting is read with `Setting::get('key')`. The value is resolved in this order, first match wins:

1. **Admin > Settings**: a value saved in the admin panel (stored in the `settings` table).
2. `.env`: only for gateway credentials (Razorpay and Google keys).
3. `config/store.php`: the built-in default for every setting.

`config/store.php` is the single source of defaults. To change what a brand-new install looks like, edit that file. To change a running store, use Admin > Settings. All settings are loaded once per request with a single query.

Note: Once a credential has been saved in Admin > Settings, changing it in `.env` has no effect. Update it in the admin panel instead.

5. Configuration Reference

5.1 Environment variables (.env)

Variable	Example / default	Purpose
APP_NAME	NovaMart	Application name
APP_ENV	local / production	Must be production on the live server. Hides test accounts, disables mock payments and the simulated Google login
APP_KEY	(generated)	Encryption key; create with <code>php artisan key:generate</code>
APP_DEBUG	true / false	Must be false in production; true shows error details including secrets
APP_URL	https://yourstore.in	Public URL; used for links, sitemap and the Google callback
APP_TIMEZONE	Asia/Kolkata	Timezone for order dates
DB_*	mysql, 127.0.0.1, 3306 ...	MySQL connection

Variable	Example / default	Purpose
SESSION_DRIVER	database	Sessions are stored in the database
CACHE_STORE / QUEUE_CONNECTION	database	No Redis needed
MAIL_*	log (local)	Configure SMTP for real e-mail
RAZORPAY_KEY_ID	rzp_live_...	Razorpay key ID (optional if set in admin)
RAZORPAY_KEY_SECRET	...	Razorpay key secret
RAZORPAY_WEBHOOK_SECRET	...	Secret used to verify Razorpay webhooks
GOOGLE_CLIENT_ID / SECRET	...	Google OAuth credentials (optional)
GOOGLE_REDIRECT_URI	(empty)	Leave empty to use APP_URL/auth/google/callback

5.2 Admin > Settings tabs

Tab	What it controls
Logo & Site Icon	Store name, text logo (prefix, highlight, subtitle) or image logo, icon initials and colours, favicon
Announcement & Menus	Top banner on/off, message and colour; header navigation links (JSON)
Search & Suggestions	Search box placeholder; trending search tags
Homepage & Banners	Show/hide each homepage section; hero carousel slides (JSON); section headings and subtitles
Trust & Assurance	The 4 assurance badges (JSON)
Footer & Contacts	Support phone/email, tagline, address, copyright text (use {year} for the current year), payment-methods text, social links
Gateway & Shipping	Razorpay on/off, keys, webhook secret, mock mode; Google Sign-In on/off and credentials; free-delivery threshold, standard and express fees

5.3 JSON settings format

Three settings take a JSON list. Invalid JSON is rejected when saving, so a typo cannot break the homepage.

```
Header menu: [{"title": "All Products", "url": "/products", "icon": "grid"}]

Hero slides: [{"badge": "UP TO 35% OFF", "title": "Home Appliances",
               "subtitle": "...", "image": "https://...",
               "button_text": "Shop Now", "button_url": "/category/home-appliances",
               "bg_gradient": "from-slate-950/95 via-slate-900/80 to-transparent"}]

Assurance: [{"title": "7-Day Replacement", "desc": "Hassle-free returns",
             "icon": "rotate-ccw"}]
```

Icon names come from the Lucide icon set (lucide.dev/icons), for example `truck`, `shield-check`, `zap`.

6. Payments (Razorpay)

6.1 Mock mode (local testing)

With **Mock Checkout Mode** on (the default), checkout shows a confirmation dialog instead of the Razorpay window and creates a paid order without charging anything. Mock mode only works when `APP_ENV` is not production; in production it is always off, even if the toggle is on.

6.2 Going live

1. Create a Razorpay account and complete KYC. In the Razorpay Dashboard open **Account & Settings > API Keys** and generate live keys.
2. Enter the Key ID and Key Secret in Admin > Settings > Gateway & Shipping (or in `.env`).
3. Turn **Mock Checkout Mode** off and make sure **Enable Online Payments** is on.
4. In the Razorpay Dashboard open **Webhooks** and add the URL `https://yourstore.in/webhook/razorpay` with the events **payment.captured**, **order.paid** and **payment.failed**. Copy the secret you choose into Webhook Secret.
5. Place a small real order and confirm it shows as **paid** in Admin > Orders.

6.3 Payment safety checks

- Every payment's signature is verified with Razorpay's SDK before the order is created.
- The payment must belong to the Razorpay order created for this customer's cart, and the amount must still match the cart total.
- Submitting the same payment twice (double click, refresh) returns the existing order instead of creating a second one.
- If Razorpay cannot be reached, checkout shows an error; it never falls back to a fake order.
- Turning **Enable Online Payments** off hides Razorpay at checkout, leaving Cash on Delivery only.

7. Google Sign-In (optional)

1. Open Google Cloud Console, create a project and configure the **OAuth consent screen** (External, app name, support e-mail).
2. Go to **APIs & Services > Credentials > Create credentials > OAuth client ID** and choose **Web application**.
3. Add the authorised redirect URI `https://yourstore.in/auth/google/callback` (for local testing also add `http://127.0.0.1:8000/auth/google/callback`).
4. Paste the Client ID and Client Secret into Admin > Settings > Gateway & Shipping > Google Sign-In, and turn it on.

Without credentials, the Google button uses a simulated sign-in on local installs, and on production shows "Google Sign-In is not configured". Turning Google login off hides the button everywhere.

8. Admin Panel Guide

Section	What you can do	Rules worth knowing
Dashboard	Revenue, orders, customers, average order value, recent orders, low-stock and top-selling products	Low stock = 10 units or fewer
Products	Create, edit, delete; price, compare-at price, stock, images, tags, specifications; Featured / Bestseller / Nova Choice flags; Active switch	Inactive products disappear from the shop and from existing carts. Discount % is calculated from the compare-at price
Categories	Create, enable/disable, delete	Names must be unique. A category that still has products cannot be deleted; disable it instead. Renaming updates its products
Orders	Search and filter; view details; change status; add courier and tracking number	Cancelling or refunding returns stock. Cancelled/refunded orders cannot be reopened
Coupons	Create, activate/deactivate, delete	Codes are stored in upper case and must be unique. Status shows Active, Inactive or Expired
Users	Search; enable/disable; grant or revoke admin	Disabled users are signed out immediately and cannot log in. You cannot change your own account here
Pages	Create and edit content pages (HTML editor), publish/unpublish, show in header or footer	URL slugs must be unique. Unpublished pages are visible to admins only
Settings	Everything in section 5.2	Changes apply immediately

9. Orders and Stock

Status	Meaning
pending	Placed with Cash on Delivery, awaiting confirmation
confirmed	Online payment verified, or confirmed by an admin
processing	Being packed
shipped / out_for_delivery	Handed to the courier (add carrier and tracking number)
delivered	Completed
cancelled / refunded	Closed; stock is returned automatically

- Placing an order checks stock, reduces it and increases the product's sales count in one database transaction. If another customer bought the last unit first, the customer sees an "out of stock" message.
- Customers can only add as many units as are in stock, and carts always show the current price.
- Order numbers look like NM-2026-AB12CD. Customers can view their order at `/orders/{order number}` and in My Account.
- Only customers with a non-cancelled order containing a product can review it, once per product.

10. Project Structure

Path	Contents
app/Http/Controllers/	Storefront controllers (Home, Shop, Search, Cart, Checkout, Account, Auth, Page, SEO, Webhook)
app/Http/Controllers/Admin/	Admin panel controllers
app/Http/Middleware/	AdminMiddleware (admin-only area), EnsureUserIsActive (signs out disabled users)
app/Models/	User, Product, Category, Order, Review, Coupon, Page, Setting
app/Services/	CartService (cart, coupons, shipping), RazorpayService, SearchIndexService
config/store.php	Defaults for every admin setting
database/migrations/	Table definitions
database/seeder/	Sample catalogue, pages, users, coupons and settings
resources/views/	Blade templates: layouts, shop, cart, checkout, account, pages, admin
routes/web.php	All routes
tests/Feature/StoreAuditTest.php	End-to-end tests for settings, checkout, payments, stock and admin rules

11. Routes Reference

Storefront

URL	Purpose
/	Homepage
/products, /category/{slug}, /search	Product listing, category listing, search results (filters and sorting)
/product/{slug}	Product page; POST /product/{slug}/review submits a review
/cart, /cart/add, /cart/update, /cart/remove, /cart/coupon/*	Cart and coupons
/checkout, /checkout/address, /checkout/place-cod	Checkout, address, Cash on Delivery
/checkout/payment/prepare, /checkout/payment/verify	Razorpay order creation and verification
/orders/{orderNumber}	Order confirmation and tracking
/account	My orders, profile, addresses (login required)
/login, /register, /logout, /auth/google	Authentication
/page/{slug}	Content pages (About, Contact, policies, FAQ)
/api/search/suggest, /api/cart/summary	JSON endpoints used by the header search and cart badge
/sitemap.xml, /robots.txt	SEO
/webhook/razorpay	Razorpay webhook (no CSRF; verified by signature)

Admin (login + admin role required)

/admin, /admin/products, /admin/categories, /admin/orders, /admin/coupons, /admin/users, /admin/pages, /admin/settings. Run `php artisan route:list` for the full list.

12. Testing

```
php artisan test
```

The feature tests run against the database configured in `.env` inside a transaction that is rolled back, so no data is changed. They expect the sample data from `php artisan migrate --seed`. They cover: settings saving and driving the storefront, invalid JSON rejection, Razorpay on/off, mock payment flow and replay protection, production payment safety, stock reservation and return, cart limits and price refresh, guest checkout, product/category/coupon/page rules, disabled users and review eligibility.

13. Production Deployment

13.1 First deployment

```
git clone <repository> /var/www/novamart && cd /var/www/novamart
composer install --no-dev --optimize-autoloader
npm ci && npm run build
cp .env.example .env # then edit: see checklist below
php artisan key:generate
php artisan migrate --force
php artisan storage:link
php artisan config:cache && php artisan route:cache && php artisan view:cache
```

Important: Do not run `db:seed` in production. It deletes users, products, orders and settings, and creates accounts with the publicly known password `Password@123`. Create the first admin with the command below instead.

```
php artisan tinker
>>> App\Models\User::create(['name' => 'Store Admin', 'email' => 'you@yourstore.in',
...   'password' => 'a-strong-password', 'role' => 'admin', 'is_active' => true]);
```

13.2 Checklist

Item	Setting
Environment	<code>APP_ENV=production</code> , <code>APP_DEBUG=false</code> , correct <code>APP_URL</code> (https)
Web server	Document root = <code>public/</code> ; HTTPS enabled
Permissions	<code>storage/</code> and <code>bootstrap/cache/</code> writable by the web server user
Database	Dedicated MySQL user with access to this database only
Payments	Live Razorpay keys, mock mode off, webhook configured (section 6.2)
E-mail	Real SMTP settings in <code>MAIL_*</code>
Queue	Run <code>php artisan queue:work</code> under Supervisor if you add queued jobs (none are required today)
Backups	Daily MySQL backup (mysqldump or managed backups)

13.3 Updating an existing installation

```
php artisan down
git pull
composer install --no-dev --optimize-autoloader
npm ci && npm run build
php artisan migrate --force
php artisan optimize:clear
php artisan config:cache && php artisan route:cache && php artisan view:cache
php artisan up
```

14. Troubleshooting

Problem	Fix
Page shows unstyled HTML	Run <code>npm run build</code> (or keep <code>npm run dev</code> running)
"No application encryption key"	<code>php artisan key:generate</code>
SQLSTATE connection refused / access denied	Check <code>DB_*</code> in <code>.env</code> and that MySQL is running; then <code>php artisan config:clear</code>
Changes to <code>.env</code> have no effect	Run <code>php artisan config:clear</code> (config is cached in production). For gateway keys, check whether a value is saved in Admin > Settings, which takes priority
Online payment option missing at checkout	Admin > Settings > Gateway: turn on Enable Online Payments
"Online payments are not configured"	Production with mock mode: enter live Razorpay keys
Google button missing / "not configured"	Turn Google login on and enter client ID and secret (section 7)
Admin redirected to login or 403	The account must have role admin and be active
Homepage section missing	Admin > Settings > Homepage & Banners: check its visibility switch
Errors after deploying new code	<code>php artisan optimize:clear</code> , then rebuild caches; check <code>storage/logs/laravel.log</code>

15. Known Limitations

- If a customer pays online and closes the browser before returning to the store, no order is created. The Razorpay webhook updates existing orders but does not create missing ones; such payments must be refunded or matched manually in the Razorpay Dashboard.
- The contact form confirms receipt but does not yet e-mail or store the message.
- Order, review and product link columns (`user_id`, `product_id`, `category_id`) are text columns without database foreign keys.
- GST/tax is not calculated (tax amount is 0); prices are treated as tax-inclusive.